



COM 114

Systems

Hardware

BSc Cs

J. Kibuthi - PhD (CS) : MSc (IT)

INPUT/OUTPUT ORGANIZATION

Objectives

By the end of this session, students should be able to:

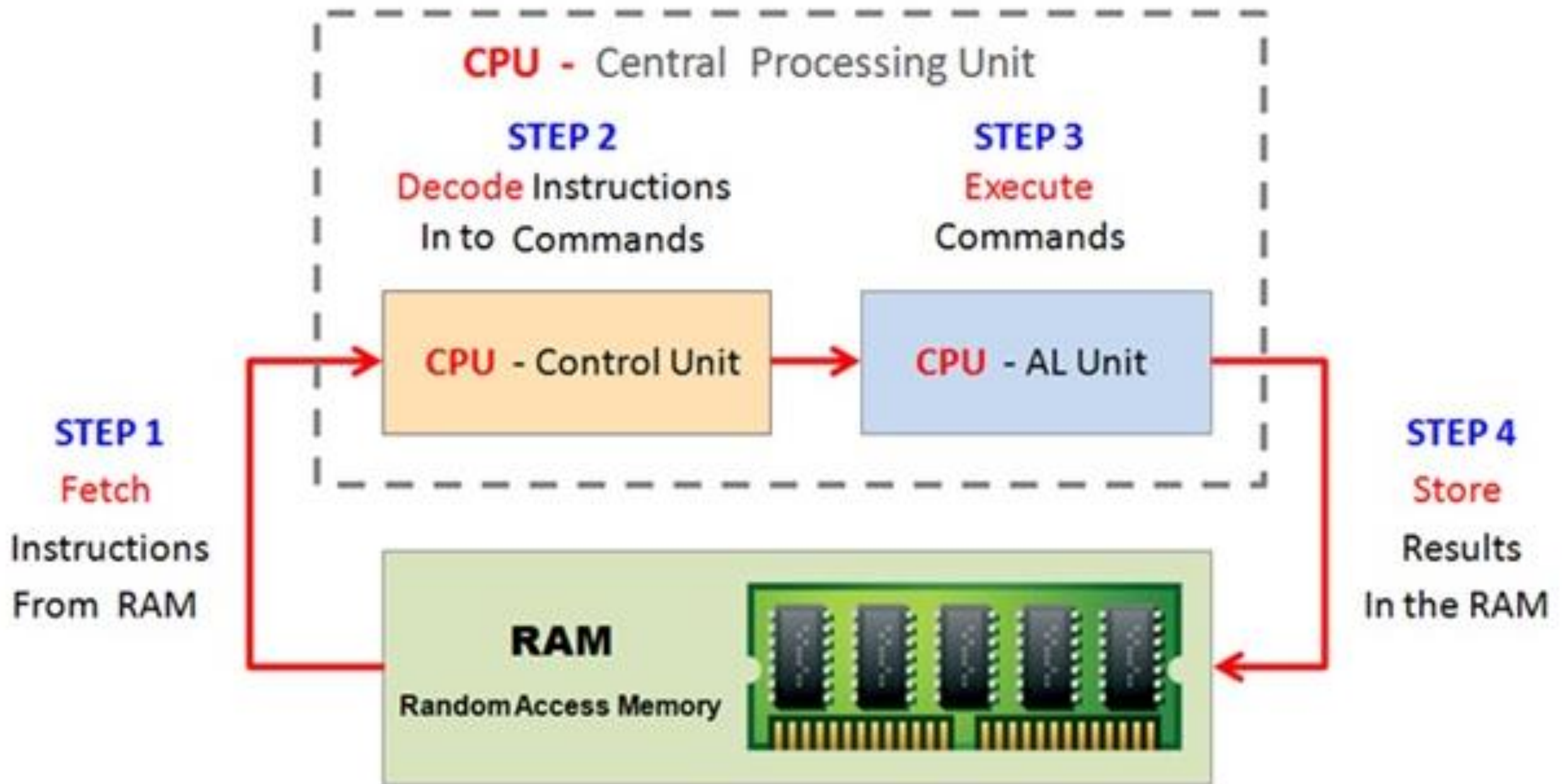
1. Define I/O organization and explain its significance.
2. Describe the fundamental components of an I/O system.
3. Explain how I/O devices interact with the CPU and memory.
4. Differentiate between programmed I/O, interrupt-driven I/O, and DMA.

- **Where?** - Identification of the device under communication – The device and location within the device Eg: Select the Printer.
- **What?** - What is the type of communication? i.e sender? Receiver? (Read/Write?)
- **How?** - How much to be communicated? (Number of bytes to/from Memory/file)

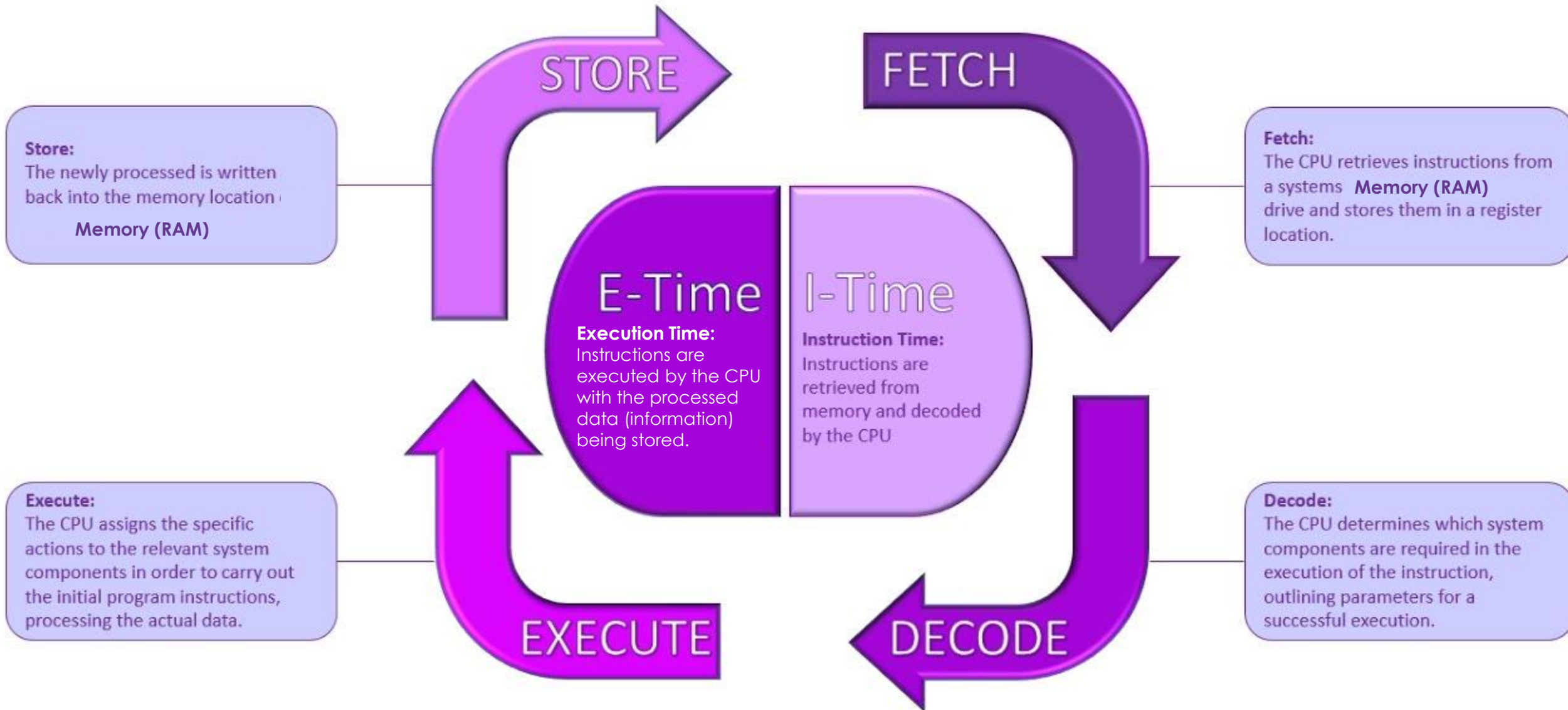
Introduction

Input/Output organization defines how I/O devices communicate with the CPU and memory. It determines how efficiently data is transferred and processed across components.

Type	Examples	Function	Transfer Characteristics
Input	Keyboard, Mouse	Capture data	Character-based, low speed.
Output	Monitor, Printer	Display data	Character/Pixel/Page-based, variable speed.
Storage	Hard Disk, SSD	Store data	Block-based, high speed/throughput
Communication	NIC, Modem	Transmit data	Packet-based, high speed.



The Fetch-Execute Cycle



Instruction Cycle

The "**fetch, decode, execute, store**" cycle is the fundamental process by which a computer's central processing unit (CPU) executes instructions, retrieving them from memory, interpreting their meaning, performing the required operation, and saving the result.

The cycle is split into two times (instruction time and execution time).

During the **Instruction time (I-time)**, instructions are retrieved from memory (RAM) and decoded by the CPU while in the **Execution time (E-time)**, program instructions are executed by the CPU with the processed data (information) being stored.

This cycle, also known as the instruction cycle, repeats continuously to run programs, involving the program counter, memory locations, control unit, arithmetic logic unit (ALU), and various registers.

Phases of the Fetch – Execution Cycle

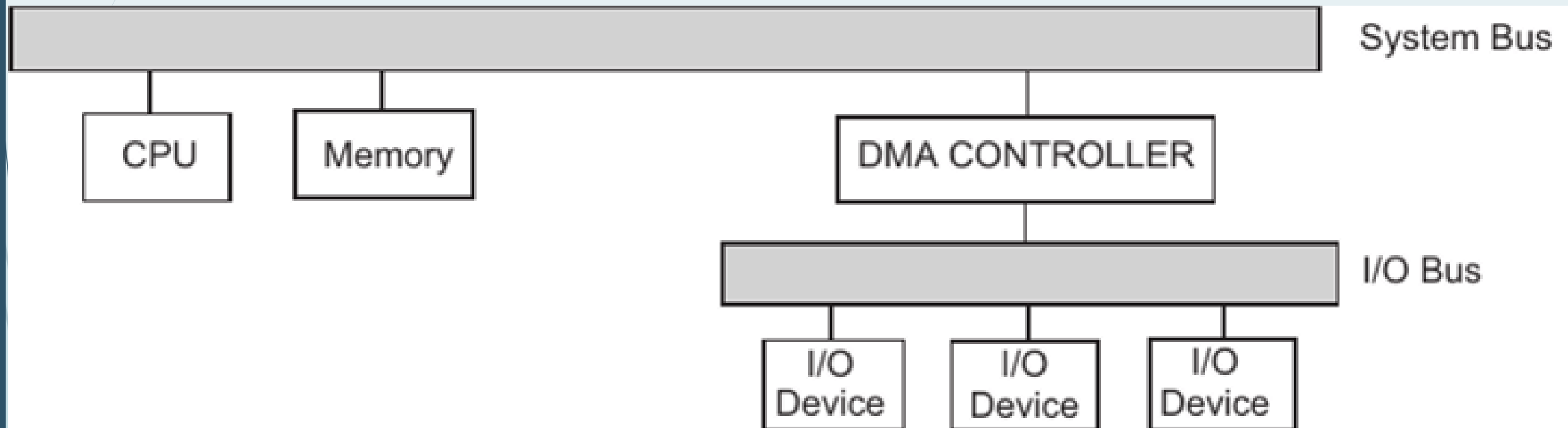
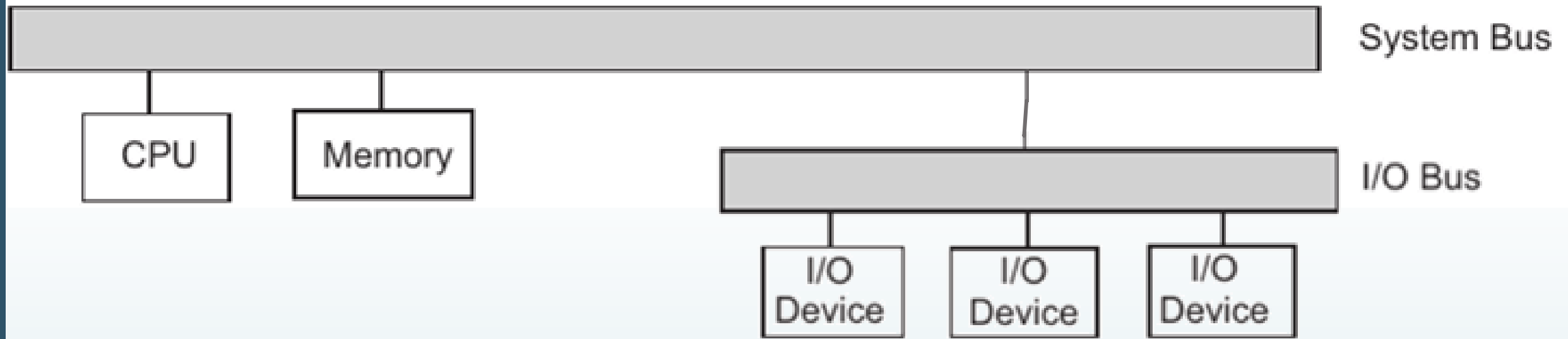
1. Fetch: The CPU uses the address stored in the program counter to locate the next instruction in memory. The instruction is then retrieved from that memory location and loaded into the CPU's current instruction register (CIR).

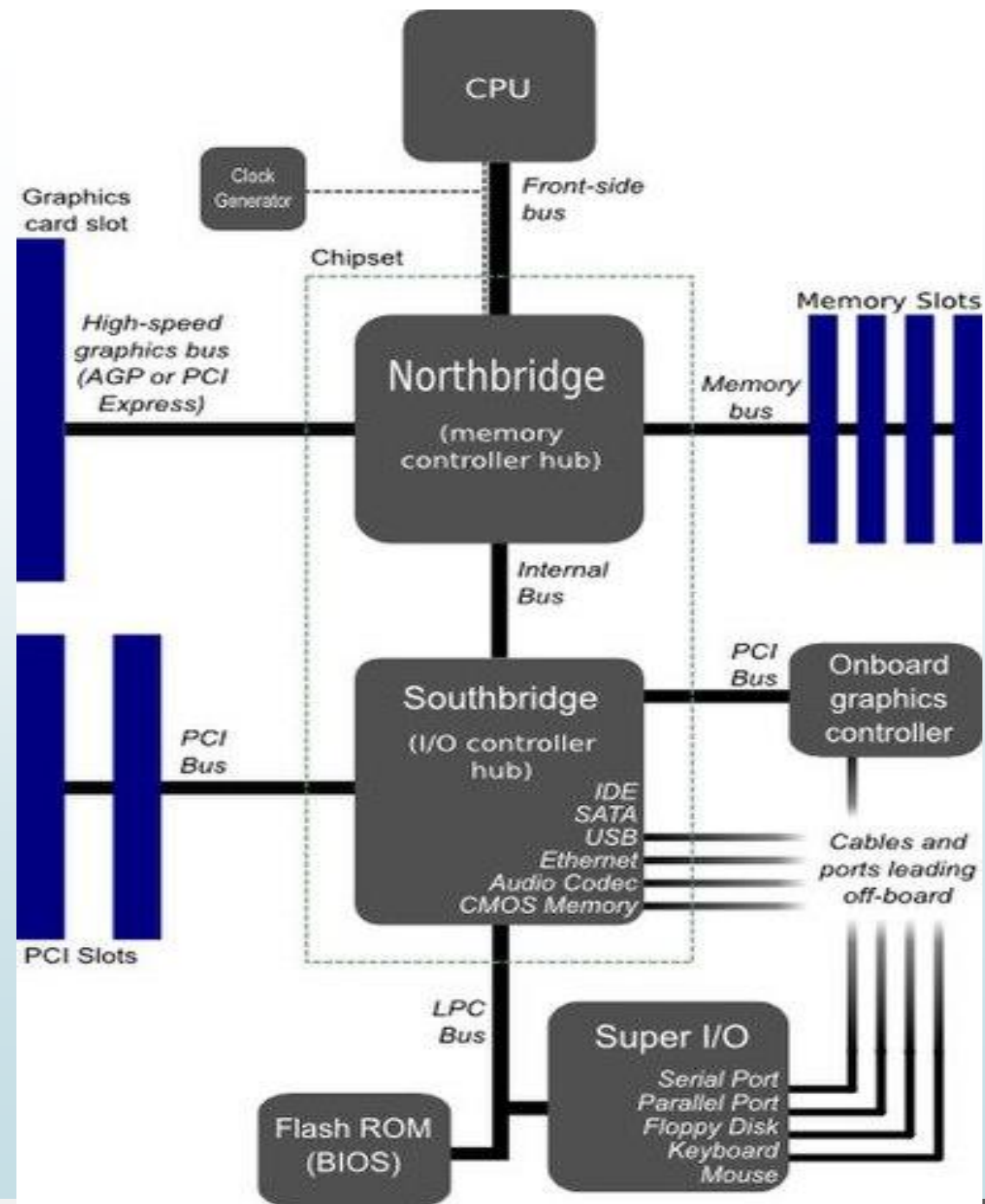
2. Decode: The control unit (CU) interprets the instruction in the CIR to understand what operation needs to be performed and which data is involved.

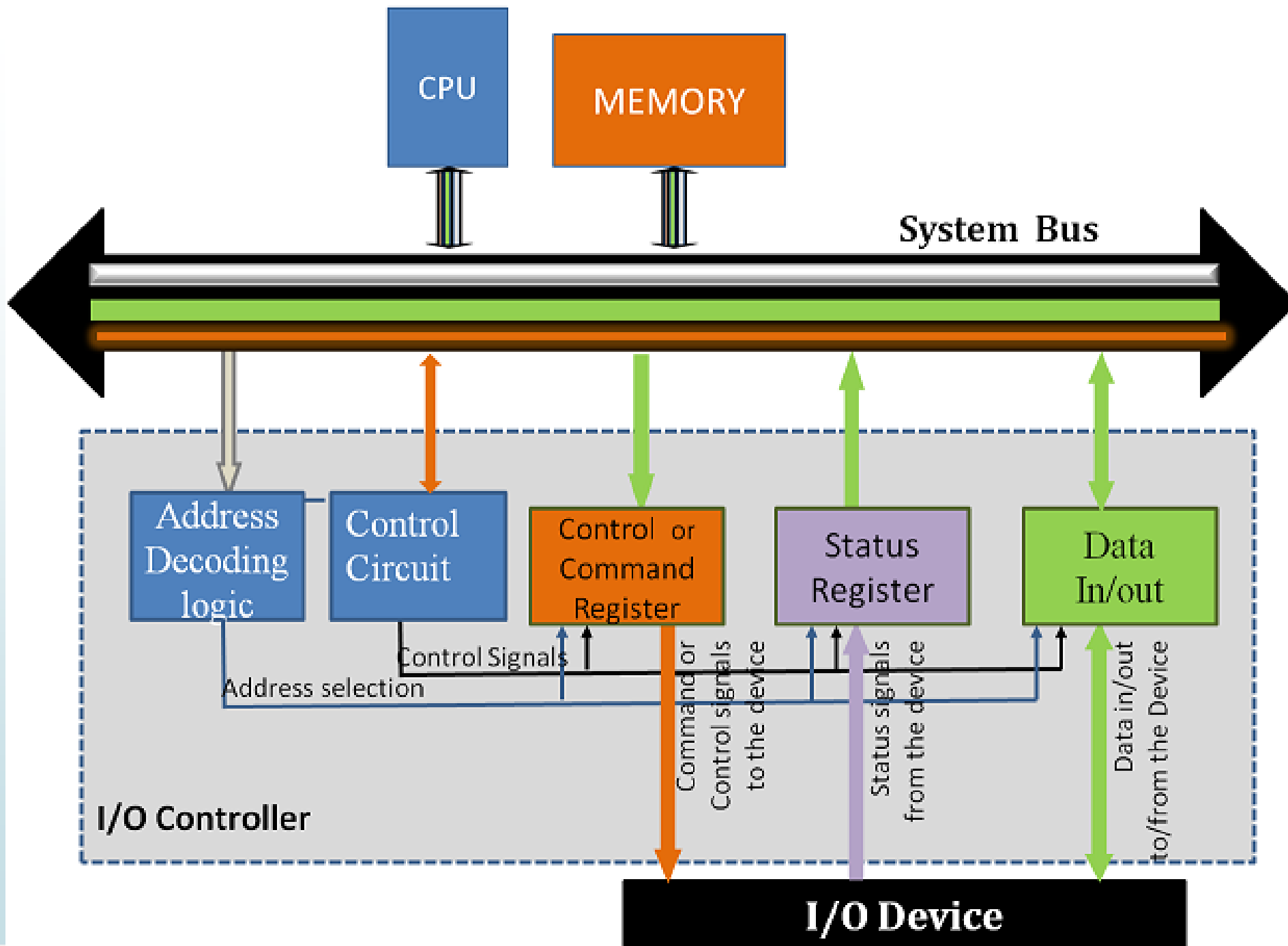
3. Execute: The arithmetic logic unit (ALU) performs the operation specified by the instruction. This could involve a calculation, moving data, or other actions.

4. Store: The result of the execution is saved back into a register or memory for future use.

The CPU goes through these steps for every single instruction in a program, repeating billions of times per second.







The **address decoding logic** is connected to the address bus. The value on the address bus indicates the register to be accessed. The decoding logic converts this as an address selection signal to one of these registers.

The **control circuitry** is connected to the control signals of the system bus. These signals are MEMW, MEMR, IOR, IOW, INTERRUPT, BREQ, etc. These signals ensure the synchronization and validation of address and Data on the bus, demanding the Bus for data transfer, sending interrupt after the normal or abnormal end operation.

The **Data Registers** take care of Data Transfer. There may be Data In and/or Data Out Registers depending on the device. Also, fast devices like Disk will have a Buffer so that the fast bulk data from the disk is stored and then sent to Memory when the system bus is available.

The **Control Register** has information like:

Which is the device to be accessed? (Address of the device)

What is the Operation (Command) to be carried out on the device?

Printer Commands – Form Feed for a new page, Print, Line Feed, status check, etc.

Disk – Seek, Write, Read, Reset, status check

How this operation is to be carried out?

Number of Bytes in case of Data Transfer

What communication is expected by CPU after data transfer to memory

Mode Settings applicable to devices, etc.

The purpose of **Status Register** is: To have the information about the device being selected

Whether the device is online and ready?

If not online have the information about the error from the device. Example - a) Printer may not have a cartridge or paper. This error status is collected in the status register. b) The Disk may not be getting ON

The status of the data transfer on the device side is stored as a flag in the Status Register.

Whether the data transfer was successful?

Data Overrun or Underrun may occur with fast devices like DISK. The buffer operates on PUSH – PULL mode. So in case of an error in PUSH or PULL side Data Overrun or Underrun occurs.

The status register keeps track of the Word Count pending to be transferred from/to Memory. A Counter is maintained in the I/O controller for this purpose. When Word Count becomes ZERO, it is implied that the data transfer is successfully done.

I/O system refers to the communication between a computer system (CPU and main memory) and the outside world. The CPU and Memory are typically **electronic** (fast, digital, small word size), while external devices (**peripherals**) are often **electromechanical** or operate at vastly different speeds (slow, analog, character/block-oriented).

Goal of I/O System is to bridge this gap efficiently and reliably by coordinating the main components of I/O system (*CPU, Memory, I/O Modules (Controllers), and Devices.*)

- **Key Functions of I/O System:**

- Data Transfer:** Moving data between peripherals and memory/CPU registers.

- Control/Command:** Sending commands (e.g., *read, write, seek*) to the I/O device.

- Status Reporting:** Allowing the CPU to check the device state (e.g., *busy, ready, error*).

- Synchronization:** Coordinating the slow I/O operations with the fast CPU.

Characteristics and Diversity of I/O Devices

A. Classifying I/O Devices

I/O devices exhibit **vast diversity** in type, speed, data rate, and control complexity.

Category	Characteristics	Examples
Human Readable	Suitable for communication with the user.	Printers, Monitors, Keyboards, Mouse
Machine Readable	Suitable for communication with other equipment.	Disk Drives, Tapes, Sensors, Actuators
Communication	Suitable for communication with remote devices.	Modems, Network Interface Cards (NICs)

Characteristics and Diversity of I/O Devices

B. Key Differences Among Devices

- Data Rate: Varies wildly, from a few bytes/sec (keyboard) to gigabytes/sec (high-speed SSD/NIC).
- Unit of Transfer:
 - ❖ Character/Byte-Oriented: Data is handled a byte at a time (e.g., keyboard, serial port).
 - ❖ Block-Oriented: Data is handled in fixed-size blocks (e.g., disk drives, network packets).
- Control Complexity: Simple devices (timer) vs. complex devices requiring sophisticated control (disk drive requiring seek and *rotational latency* compensation).
- Error Conditions: Varies from simple electrical faults to complex media errors.

C. The I/O Module (I/O Controller)

Since the CPU cannot directly manage every diverse device, a specialized electronic circuit, the **I/O Module (or Device Controller)**, acts as an intermediary.

• Functions of the I/O Module:

1. Control and Timing: Coordinating traffic between internal resources and external devices.
2. CPU Communication: Decodes commands, reports status, transfers data.
3. Device Communication: Implements the specific interface protocol for the peripheral.
4. Data Buffering: Temporarily holds data to match speed differences.
5. Error Detection: Detecting and reporting device errors.

3. I/O Bus and Interconnection Structure

A. The Computer Bus Structure

- A Bus is a shared communication pathway connecting multiple components. It consists of multiple lines (wires).
 - Address Lines: Specify the source or destination of the data (memory location or I/O port).
 - Data Lines: Carry the actual information being transferred.
 - Control Lines: Transmit control and timing signals (e.g., *read/write*, *bus request*, *clock*).

B. Hierarchical Bus Architecture (Modern Systems)

To accommodate the high speed of the CPU/Memory and the diverse speeds of I/O devices, a multi-level, hierarchical bus structure is used.

- 1. System Bus (or Processor-Memory Bus):
 - High-Speed: Optimized for maximum throughput between the CPU and the fastest level of cache/memory.
 - Narrow Focus: Only handles CPU/Memory traffic.

- **2. High-Speed I/O Bus (e.g., PCIe - Peripheral Component Interconnect Express):**

- **Bridge:** Connects the system bus to fast peripherals (e.g., graphics cards, high-speed NICs, SSDs).
- **Protocol:** Uses a dedicated, high-speed, serial, point-to-point connection (unlike the shared parallel System Bus of old).
- **3. Low-Speed I/O Bus (e.g., USB, SATA, older PCI):**
 - **Controller:** Connected to the High-Speed I/O Bus (via a controller/hub).
 - **Purpose:** Connects slower, less demanding peripherals (e.g., keyboard, mouse, slower drives, audio).

C. Interfacing I/O Devices to the Bus

- **I/O Port:** A register or set of registers within the I/O Module that the CPU can read from or write to.
- **The I/O Module's Role:** It translates high-level bus signals (from the CPU) into the control, status, and data signals required by the specific I/O device.

- **2. High-Speed I/O Bus (e.g., PCIe - Peripheral Component Interconnect Express):**

- **Bridge:** Connects the system bus to fast peripherals (e.g., graphics cards, high-speed NICs, SSDs).
- **Protocol:** Uses a dedicated, high-speed, serial, point-to-point connection (unlike the shared parallel System Bus of old).
- **3. Low-Speed I/O Bus (e.g., USB, SATA, older PCI):**
 - **Controller:** Connected to the High-Speed I/O Bus (via a controller/hub).
 - **Purpose:** Connects slower, less demanding peripherals (e.g., keyboard, mouse, slower drives, audio).

C. Interfacing I/O Devices to the Bus

- **I/O Port:** A register or set of registers within the I/O Module that the CPU can read from or write to.
- **The I/O Module's Role:** It translates high-level bus signals (from the CPU) into the control, status, and data signals required by the specific I/O device.

CPU-I/O Interconnection Methods

1. Overview of I/O Control Techniques

The primary goal is to manage the flow of data between the I/O device and the main system components (CPU/Memory). There are three main techniques:

- 1. Programmed I/O (PIO):** The CPU directly controls the I/O operation and waits for its completion. *CPU is heavily involved.*
- 2. Interrupt-Driven I/O:** The CPU issues an I/O command and then continues with other tasks; the I/O module interrupts the CPU upon completion. **CPU is moderately involved.**
- 3. Direct Memory Access (DMA):** The I/O module (DMA Controller) controls the data transfer directly to/from memory without CPU intervention. **CPU is minimally involved.**

Method	Description	Example	Cons	Limitations
Programmed I/O	CPU polls device status.	Keyboard input	Simple	CPU time wasted
Interrupt-Driven I/O	Device interrupts CPU when ready.	Mouse	Efficient CPU use	Needs interrupt handling
DMA	Transfers data directly between device and memory.	Disk I/O	Fast, efficient	Complex controller

2. Programmed I/O

A. Execution Cycle

1. CPU sends a command to the I/O module (e.g., *READ*).
2. CPU continually checks the status register of the I/O module (a process called **Polling** or **Busy-Waiting**).
3. When the status indicates the device is *ready*, the CPU reads the data from the I/O module's data register and writes it to memory (or vice-versa).
4. This process is repeated for every **word** or **byte** of data.

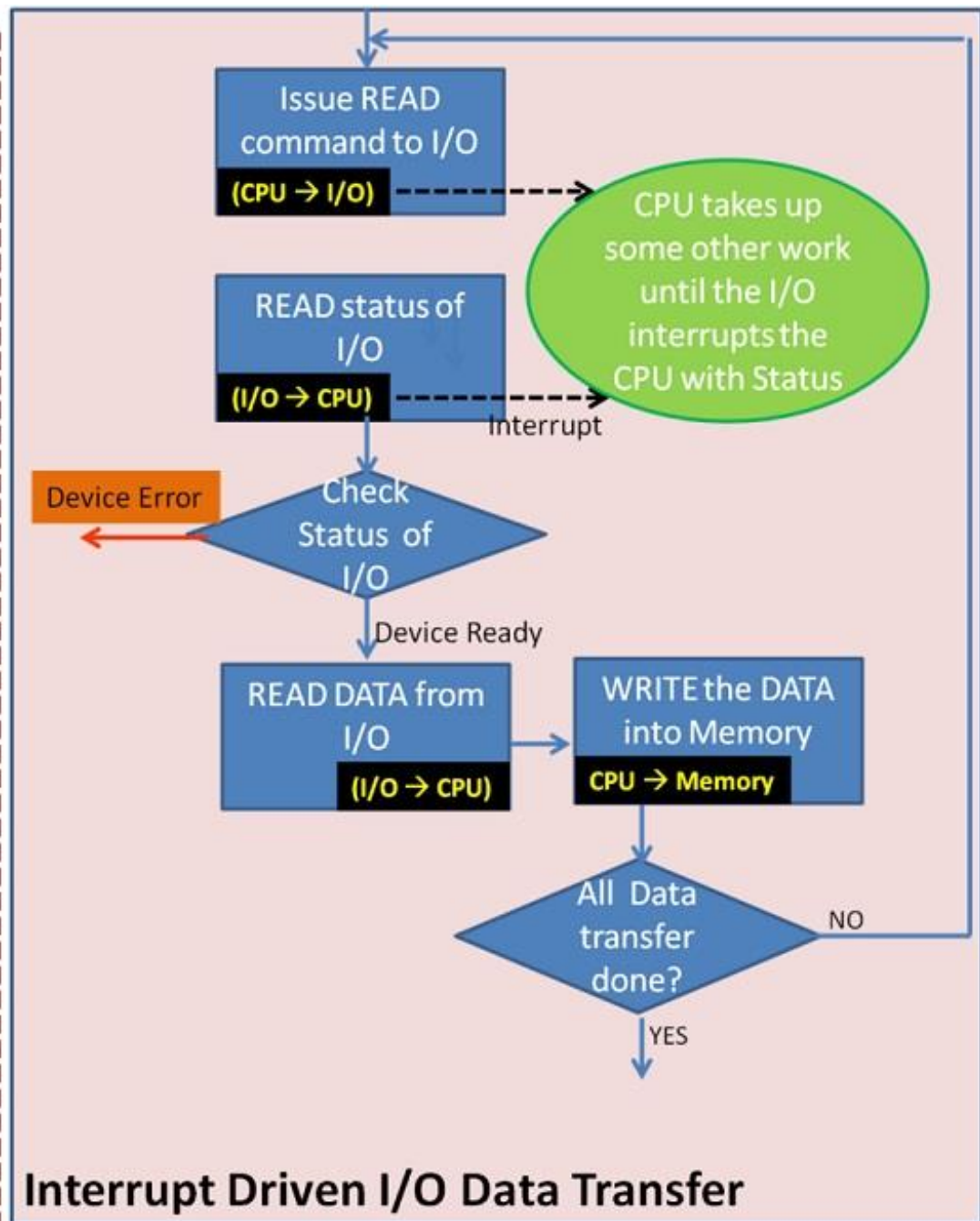
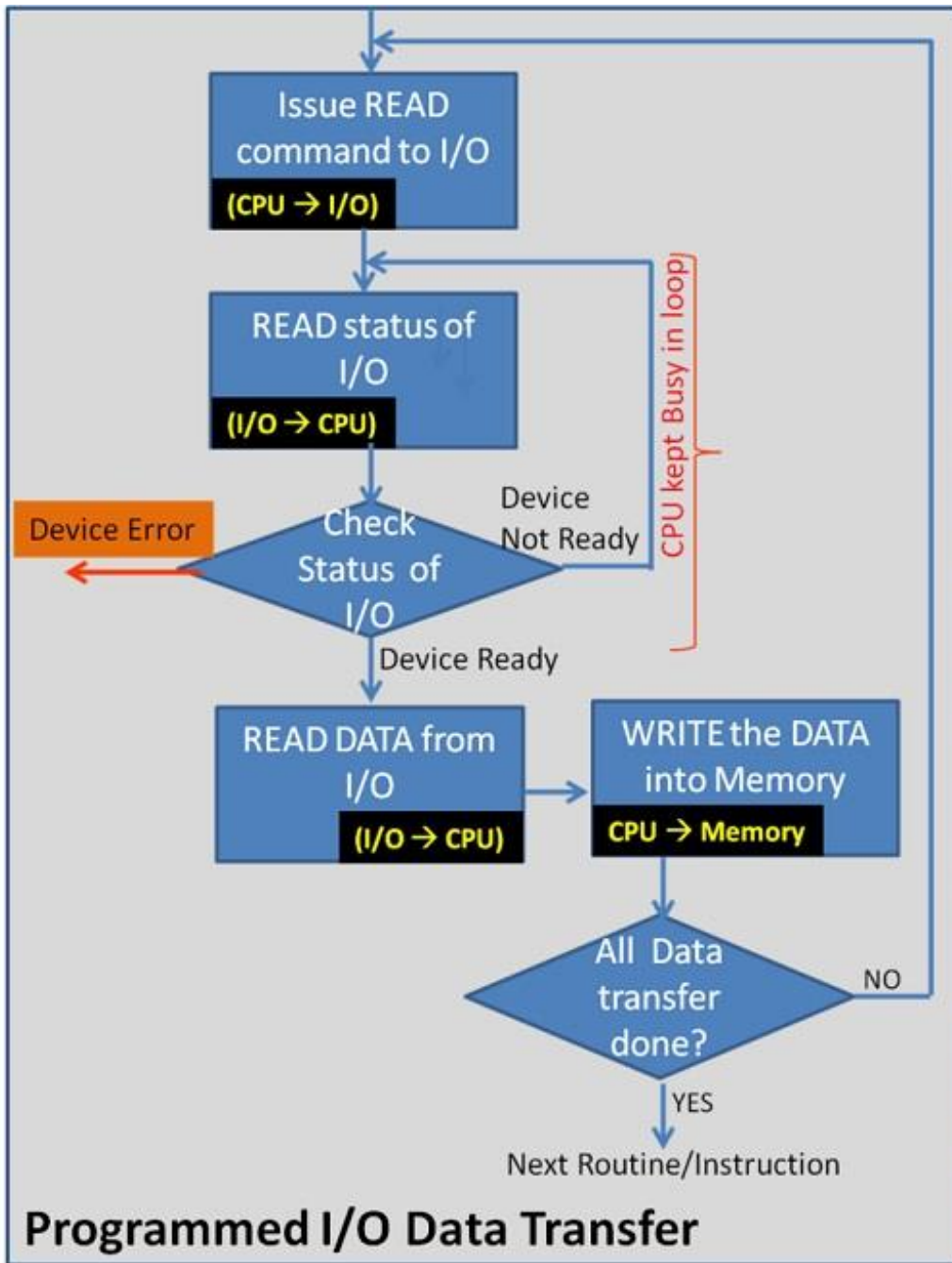
B. Addressing I/O Devices

1. Memory-Mapped I/O:
 - Concept: I/O device registers (ports) are assigned specific addresses within the main memory address space.
 - **Advantage:** Uses the same instructions for memory and I/O (e.g., *LOAD*, *STORE*).

- Disadvantage: Reduces the available memory address space.
- 2. Isolated I/O (or Port-Mapped I/O):
 - Concept: I/O devices have a separate address space (I/O Port space) distinct from memory.
 - Advantage: Does not consume memory address space.
 - Disadvantage: Requires specific, dedicated I/O instructions (e.g., IN, OUT) and logic on the bus.

C. Drawback of PIO

- **Inefficiency:** The CPU spends a huge amount of time **waiting** or **polling** for the slow I/O device, severely wasting CPU cycles. This is only suitable for very low-speed or infrequent I/O.



3. Interrupt-Driven I/O

A. Execution Cycle

1. CPU issues an I/O command to the module.
2. CPU immediately continues executing other instructions.
3. When the I/O module finishes the operation, it generates a hardware Interrupt Signal on the control bus.
4. CPU detects the interrupt and transfers control to the Interrupt Service Routine (ISR).
5. The ISR handles the data transfer (e.g., one block/buffer of data) and clears the interrupt.
6. The CPU returns to the interrupted program.

B. Interrupt Handling

- Identification: The CPU must determine which device caused the interrupt. Methods include:
 - Multiple Interrupt Lines: A dedicated line for each device (impractical for many devices).
 - Software Poll: The ISR polls the status registers of each device to find the one requesting service.
 - Daisy Chain: Devices are wired serially; the device closest to the CPU has the highest priority.
 - Bus Arbitration/Vectored Interrupts: The I/O module places a unique identifier (the Interrupt Vector) on the data bus, allowing the CPU to jump directly to the correct ISR.

C. Advantage

- Improved Efficiency: The CPU is freed from busy-waiting and can perform useful work while the I/O operation is in progress.

3. Direct Memory Address

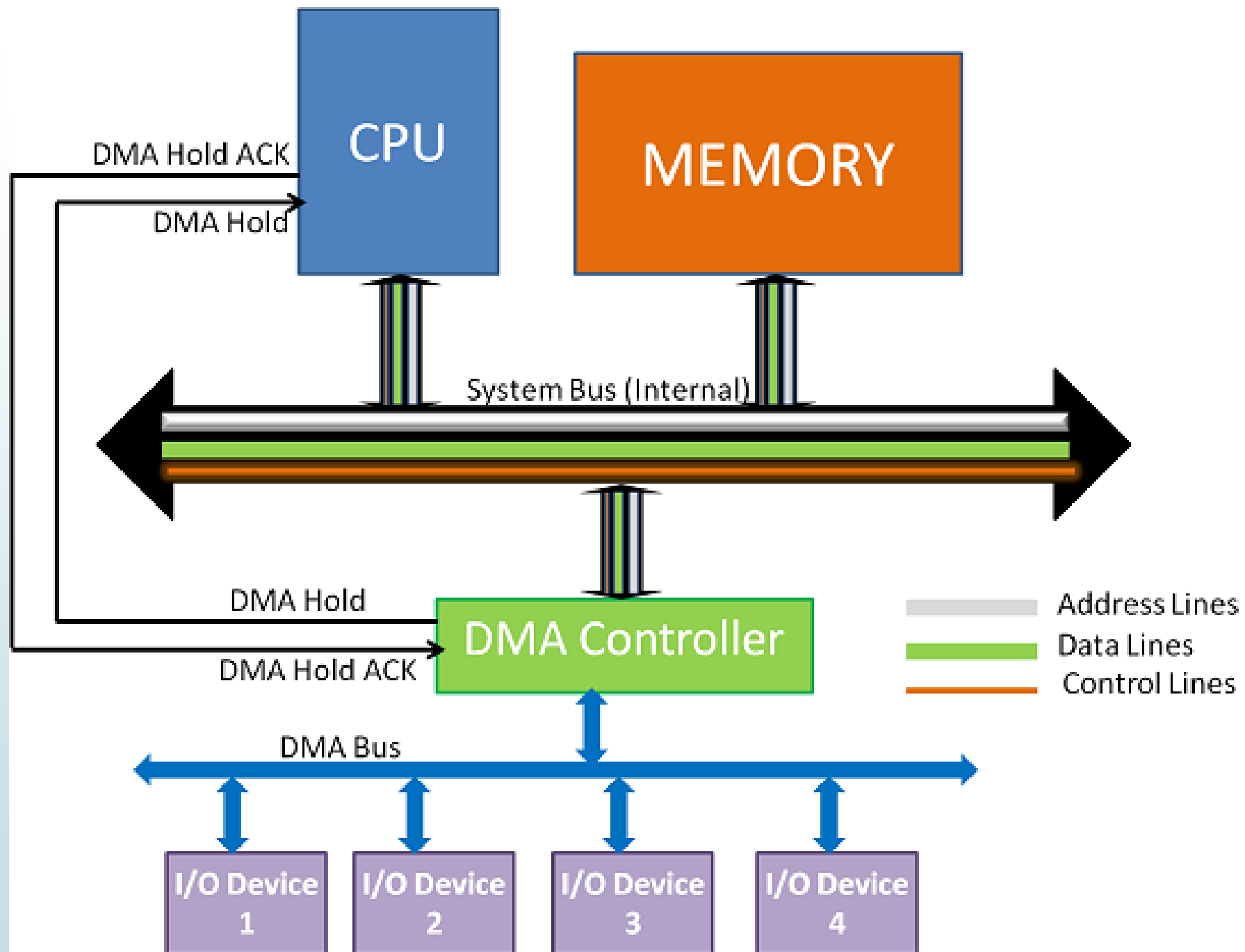
A DMA controller facilitates data transfer between I/O and Memory directly, instead of involving CPU as in the other two cases of I/O data transfer.

(Programmed I/O and Interrupt-Driven I/O)

DMA allows devices to move data directly to/from memory without per-word CPU involvement, significantly reducing CPU workload for bulk transfers. DMA is highly efficient for large block transfers (e.g., disk I/O).

The CPU programs the DMA controller with source/destination addresses, transfer size, and direction.

The DMA controller arbitrates for the bus, performs the transfer, and interrupts the CPU when complete.



DMA controllers are used widely:

Storage controllers (NVMe SSDs), NICs (zero-copy packet transfer), GPUs (bulk texture uploads), and audio/video pipelines.

DMA descriptors and scatter-gather lists allow complex transfers without CPU per-word intervention.

Implementation details (e.g., IOMMU) matter for security; IOMMU remaps device DMA addresses and prevents rogue device access to arbitrary physical memory.

High-level fact: DMA dramatically reduces CPU overhead for block transfers; it's essential for high-throughput network and storage subsystems